

SAÉ 2.02 – Exploration algorithmique d'un problème

Rapport de projet

SCHELTIEN Mattéo et LEPAGE Thomas (Groupe S2 A)
31-05-2026

Sommaire

1. Présentation du projet	2
2. Représentation d'un graphe	3
3. Calcul du plus court chemin via l'algorithme du point fixe	4
4. Calcul du plus court chemin via l'algorithme de Dijkstra	5
5. Application aux données du réseau STAN	6
5.1 Comparaison des temps de calcul des algorithmes de Bellman-Ford et Dijkstra	6
5.2 Comparaison des chemins obtenus par notre application et le moteur de recherche du réseau STAN	7
6. Conclusion	8

! Informations du projet

- Nom/Prénom/Groupe des membres du binôme :
 - Thomas LEPAGE S2A
 - Mattéo SCHELTIEN S2A
- URL du dépôt git : https://github.com/ThomasLpg/S2A_SCHELTIEN_LEPAGE.git
- Classe contenant votre méthode main : `./MainTransport.java`
- Commande à saisir pour compiler le fichier jar :
 - `jar cfe Projet.jar Main Main.class`

1. Présentation du projet

Note

Présenter votre application dans les grandes lignes :

- ce à quoi elle sert,
- ce qu'elle prend en entrée et retourne en sortie,
- comment elle fonctionne (ce qu'il se passe quand on exécute la méthode `main`),
- comment elle est structurée (quelles classes sont utilisées, éventuellement quels paquets),
- comment elle a été évaluée (sur quels critères, avec quelles données)

Dans ce projet, nous avons représenté un graphe orienté pondéré et appliqué le calcul du plus court chemin entre deux sommets.

Pour cela, nous avons implémenté les algorithmes de Bellman-Ford et Dijkstra.

Le but final est de pouvoir appliquer ces algorithmes au réseau de transport STAN afin de les performances des deux méthodes dans la recherche du meilleur chemin.

Notre programme prend en entrée un fichier représentant un graphe avec des nœuds et des arêtes, puis il le construit et le garde en mémoire. Ensuite, il calcule les plus courts chemins à partir d'un sommet de départ vers un sommet cible.

En sortie, le programme affiche les chemins les plus courts et leurs coûts.

Le projet est structuré dans ces différentes classes :

- représentation du graphe (`GrapheListe`, `Arc`, `Arcs`)
- algorithmes (`BellmanFord`, `Dijkstra`)
- gestion des valeurs (`Valeurs`)
- lecture des données (`LireReseau`)
- classes de test et d'exécution (`Main`, `MainTransport`, etc.)

2. Représentation d'un graphe

Note

Expliquez en quelques lignes comment sont représentés les graphes :

- quelles structures de données sont utilisées ?
- quelles opérations sur ces structures sont autorisées ?
- sur quels exemples ces structures ont-elles été développées ?
- si vous avez défini des tests unitaires, que font-ils ?

NB: pour chacune de ces questions, vous indiquerez les fichiers et numéros de ligne correspondants dans votre code. Vous pouvez par ailleurs utiliser des diagrammes (par exemple diagramme UML de classe) pour illustrer vos propos.

Nous représentons nos graphes sous forme de liste d'adjacence, chaque sommet est stocké dans une liste, et chaque sommet possède une liste d'arcs sortants.

Fichier : Graphe.java pour l'interface

Classe Arcs fichier Arcs.java

Elle représente une liste d'arcs.

Elle a comme attributs :

- listeArcs : ArrayList d'Arc (ligne 5)

Elle a comme méthodes :

- ajout d'un arc (ligne 12)
- un getter (ligne 16)
- toString() (ligne 24)

Classe GrapheListe fichier GrapheListe.java

Elle implémente l'interface Graphe

Structure :

- ArrayList pour stocker les nœuds (ligne 13)
- ArrayList pour stocker les listes d'adjacence (ligne 18)
- classe Arc pour représenter une arête avec un poids
- **Classe Arc** fichier Arc.java

Elle représente une arête du graphe.

Elle a comme attributs :

- noeudCible : chaîne de caractères représentant le sommet cible (ligne 2)
- poids : entier représentant le coût de l'arête (ligne 3)

Elle a comme méthodes :

- des getters (ligne 15 ; 19)
- toString() (ligne 23)

3. Calcul du plus court chemin via l'algorithme du point fixe

Note

Expliquez en quelques lignes comment a été implémenté l'algorithme du point fixe :

- quelles structures de données ont été utilisées ?
- comment ont elles été transformées au cours de l'algorithme ?
- sur quels exemples ces structures ont elles été développées ?
- si vous avez défini des tests unitaires, que font ils ?

NB: pour chacune de ces questions, vous indiquerez les fichiers et numéros de ligne correspondants dans votre code. Vous pouvez par ailleurs utiliser des diagrammes (par exemple diagramme UML de classe) pour illustrer vos propos.

L'algorithme de Bellman-Ford est implémenté dans la classe BellmanFord.

La méthode principale `resoudre` prend en paramètre un graphe et un sommet de départ.

Elle initialise :

- toutes les distances à +infini (`Double.MAX_VALUE` ligne 11)
- le sommet de départ à 0 (ligne 14)
- les sommets parents à null (ligne 12)

Structure :

- classe Valeurs pour stocker distances et parents (fichier Valeurs.java)
- listes d'arcs du graphe (ligne 19)

Évolution des données :

À chaque entrée dans la boucle :

- les distances sont mises à jour si un chemin plus court est trouvé (ligne 26-28)
- les parents sont mis à jour pour reconstruire le chemin (ligne 29)

Tests : fichier TestBellmanFord.java

- cohérence des chemins (19-22)
- cas simples (chemin vers soi-même, etc.)

4. Calcul du plus court chemin via l'algorithme de Dijkstra

Note

Expliquez en quelques lignes comment a été implémenté l'algorithme de Dijkstra :

- quelles structures de données ont été utilisées ?
- comment ont elles été transformées au cours de l'algorithme ?
- sur quels exemples ces structures ont elles été développées ?
- si vous avez défini des tests unitaires, que font ils ?

NB: pour chacune de ces questions, vous indiquerez les fichiers et numéros de ligne correspondants dans votre code. Vous pouvez par ailleurs utiliser des diagrammes (par exemple diagramme UML de classe) pour illustrer vos propos.

L'algorithme de Dijkstra est implémenté dans la classe Dijkstra.

Elle s'appuie sur un Graphe (GrapheList), expliqué précédemment, implémente une « ArrayList » de chaîne de caractère (q) qui stocke les sommets restant à traiter et s'appuie sur la classe Valeur, expliqué précédemment.

La classe Valeurs est initialisé avec toutes les distances au maximum de la valeur qu'un double peut prendre (Double.MAX_VALUE) et les parents à null (sauf le nœud de départ qui est à 0.0). La liste q se vide à chaque itération et les dictionnaires (treeMap) se mettent à jour. Lorsque le che-min le plus court vers un nœud voisin qui est encore dans q est trouvé, sa distance est mis à jour par la valeur minimal pour l'atteindre depuis le nœud de départ et sont attribut parent est remplacé par le nœud qui était dans l'itération.

- ArrayList<String> pour la file de traitement (ligne 15)
- Valeurs pour les distances et parents (ligne 12)
- TreeMap pour organiser les distances fichier (Valeurs.java)

Initialisation :

- toutes les distances sont initialisées à Double.MAX_VALUE (ligne 21)
- le sommet de départ est initialisé à 0 (ligne 30)
- les parents sont initialisés à null (ligne 23)

Fonctionnement :

- on récupère le sommet de distance minimale (ligne 39-44)
- on met à jour ses voisins (56-72)
- on met à jour les distances et les parents si un meilleur chemin est trouvé (62-69)

Tests unitaires : fichier TestDijkstra.java

Dans TestDijkstra, nous avons vérifié :

- le plus court chemin (ligne 7)
- le cas d'un sommet vers lui-même (distance 0) (ligne 29)
- les cas de sommets inatteignables (distance infinie) (ligne 45)

5. Application aux données du réseau STAN

Note

Expliquez en quelques lignes comment ont été traitées les données du réseau STAN :

- comment les données ont-elles été chargées ?
- quelles sont les caractéristiques (nombre de nœuds et d'arcs) du graphe obtenu ?
- comment les algorithmes implantés ont-ils été invoqués dans votre programme ?
- quels chemins (couples départ/arrivée) avez-vous choisis pour vos tests ? Pourquoi ?
- quels temps de calcul avez-vous obtenus ? Remplir le tableau ci-dessous.
- quel environnement logiciel et matériel avez-vous utilisé ? (processeur, mémoire vive, version du système d'exploitation, du langage Java)
- si vous avez défini des tests unitaires, que font-ils ?
- quels trajets avez-vous comparés avec le moteur de recherche du réseau STAN ?
- avez-vous obtenu les mêmes résultats ? (Indiquez la liste des arrêts donnée par votre application et par le moteur de recherche du réseau STAN, vous pouvez ajouter des captures d'écran)

NB: pour chacune de ces questions, vous indiquerez à quelle ligne de votre code votre correspond. Vous pouvez par ailleurs utiliser des diagrammes (par exemple diagramme UML de classe) pour illustrer vos propos.

Les données du réseau STAN sont chargées via la classe LireReseau, à partir des fichiers :

- stan.nodes.txt
- stan.edges.txt

Graphe : fichier Graphe.java

Le graphe obtenu contient un grand nombre de nœuds et d'arcs

Utilisation des algorithmes : fichier MainTransport.java

Les algorithmes BellmanFord et Dijkstra sont appelés depuis MainTransport, ils sont appliqués sur différents couples de stations afin de comparer les chemins obtenus et les temps de calculs

Choix des tests :

Nous avons choisi des trajets :

- trajets courts (stations proches)
- trajets moyens
- trajets longs (traversée du réseau)

Cela permet de comparer les résultats des algorithmes selon la taille du graphe.

Environnement :

- Java (version utilisée dans le projet)
- IntelliJ IDEA, VSCode
- Matériel : 13th Gen Intel® Core™ i5-1334U CPU, 16.0 GB de mémoire vive, windows 11, java version : openjdk 25.0.1 2025-10-21 LTS OpenJDK Runtime Environment Temurin-25.0.1+8 (build 25.0.1+8-LTS) OpenJDK 64-Bit Server VM Temurin-25.0.1+8 (build 25.0.1+8-LTS, mixed mode, sharing)

5.1 Comparaison des temps de calcul des algorithmes de Bellman-Ford et Dijkstra

départ	arrivée	chemin	temps calcul Bellman-Ford	temps calcul Dijkstra
HLRTT0	SCHCE1	HLRTT0;HLRTN1;JRSNG2;JRMGG1;JRCLM0;JRGAR0;JRALS0;NYLEV0;NYSAV0;NYLOB2;NYMOL0;NYLOR1;NYSPL0;NYS TD1;TMJAU3;TMFRA0;ESMZM7;ESKLE0;ESKLB1;ESCLN1;ESFYR1;ESONC0;ESSBL3;PLREN0;PLCHG1;PLBOR1;PLCES1;PLSEM1;SCROU0;SCPRA0;SCHOH0;SCGLB0;SCSUR1;SCHCE1	804331500 ns	48050200 ns

départ	arrivée	chemin	temps calcul Bellman-Ford	temps calcul Dijkstra
NYGAR1	VLJLR1	NYGAR1;NYKEN1;NYCOM0;NYCOM1;NYPAI1;NYCLG1;LXZOL1;LXFCH2;VLLIB1;VLFRA1;VLMNG1;VLARM0;VLJLR2;VLJLR1	667621100 ns	29271300 ns

départ	arrivée	chemin	temps calcul Bellman-Ford	temps calcul Dijkstra
NYVOS2	LNMON0	NYVOS2;NYQBA0;NYVIL0;NYVLL0;NYCMS1;JRTE L1;JRCLM0;JREVR0;JRFN T1;JRMSF0;JRMON0;JREMM0;LNMON0	679470600 ns	23424800 ns

Nous pouvons conclure d’après nos tests que l’algorithme de Dijkstra est beaucoup rapide quant à la résolution des chemins les plus court.

6. Conclusion

Note

Décrivez en quelques lignes le bilan de votre projet :

- rappelez ce que vous avez fait,
- donnez quelques caractéristiques de votre code (par exemple nombre de classes, nombre de méthodes, nombre de lignes de code, nombre de tests unitaires),
- décrivez les résultats de votre étude comparative (quel algorithme marche le “mieux” ? Pour quelle différence de performances en moyenne ? Pour quel type de graphe le cas échéant ?),
- précisez les qualités et limites de votre travail (par exemple, ce qui pourrait être amélioré),
- indiquez ce qui vous a paru difficile,
- proposez une notation pour votre travail en respectant le barème fourni dans l’énoncé et en indiquant pour chaque critère, une phrase précisant pour quelle raison vous avez choisi ce score (cette notation est bien entendue informative, et ne sera pas liée à l’évaluation de votre enseignant-e).

Dans ce projet, nous avons implémenté deux algorithmes de recherche de plus courts chemins, Bellman-Ford et Dijkstra.

Nous avons structuré notre code autour d'un graphe fait en liste d'adjacence et de classes permettant de gérer les arêtes et les sommets.

Le projet est représenté par :

- plusieurs classes de structure de données
- deux implémentations d'algorithmes
- des classes de tests
- une classe de chargement des données STAN

Les résultats montrent que Dijkstra est plus rapide dans la plupart des cas

Difficultés rencontrées :

- gestion des mises à jour des distances
- méthode ajoutArc de la classe GrapheListe
- écriture des deux algorithmes